

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I Sadhana S (192524298)_(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SADHANA S

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments

- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. AI-Powered Drone for Emergency Medicine Delivery

i. Greedy Best-First Search (GBFS)

Greedy Best-First Search selects the next node based only on the **heuristic value $h(n)$** (estimated distance to the goal).

- Uses **$f(n) = h(n)$** .
- Chooses the path that appears closest to the destination.
- Ignores the actual travel cost already incurred.

Strengths

- Very fast decision making.
- Requires fewer node expansions.
- Suitable for emergency situations where quick response is important.

Weaknesses

- May choose unsafe or blocked routes.
- Does not guarantee the shortest or safest path.
- Can produce suboptimal solutions if the heuristic is inaccurate.

ii. A* Search

A* considers both:

- **$g(n)$** = Actual cost from the start.
- **$h(n)$** = Estimated cost to the goal.

Formula:

$$f(n) = g(n) + h(n)$$

Working

- Calculates total estimated cost.
- Expands the node with the smallest $f(n)$.
- Continuously balances travel cost and estimated remaining distance.

Advantages

- Finds the optimal path if the heuristic is admissible.
- Avoids unnecessarily expensive routes.
- Better suited for changing movement costs.

iii. Impact of Heuristic Accuracy

Greedy Best-First Search

- Accurate heuristic → Fast and good routes.
- Poor heuristic → May follow blocked or risky paths.

A*

- Accurate heuristic → Faster search with optimal solution.
- Inaccurate but admissible heuristic → Still optimal but slower.
- Overestimating heuristic → May lose optimality.

iv. Effect of Dynamic Changes (Blocked Paths)

Dynamic obstacles require the algorithm to recalculate routes.

Greedy Search

- Frequently selects blocked paths.
- Requires repeated replanning.
- Performance decreases.

A*

- Recalculates using updated costs.
- Finds an alternative optimal path.
- More reliable in dynamic environments.

v. Recommended Algorithm

A Search*

Justification

- Produces optimal paths.
- Considers actual travel cost and heuristic.
- Handles changing terrain costs effectively.
- Provides safer navigation.
- Suitable for emergency medicine delivery where safety is critical.

2. Smart City Traffic Signal Optimization

Problem Formulation

Objective

Minimize:

- Waiting time
- Fuel consumption
- Traffic congestion

Why Traditional Search is Inefficient

Traditional search explores a huge number of signal combinations.

Problems:

- Extremely large search space.
- High computational cost.
- Traffic changes continuously.
- Slow response.

i. Hill Climbing

Hill Climbing starts with an initial traffic signal plan.

Steps:

1. Evaluate current solution.
2. Generate neighboring solutions.
3. Move to the better solution.
4. Repeat until no improvement exists.

Advantages

- Simple.
- Fast.
- Low memory.

Limitations

- May stop before reaching the best solution.
- Cannot escape local optimum.

ii. Local Maxima, Plateaus and Ridges

Local Maximum

Traffic improves slightly but is not globally optimal.

Example:

A signal timing works well for one area but increases congestion elsewhere.

Plateau

Many neighboring solutions have equal performance.

Example:

Changing signal timings produces no noticeable improvement.

Ridge

The best solution requires multiple simultaneous changes.

Example:

Changing one signal alone has no benefit, but changing several together improves traffic.

iii. Simulated Annealing and Genetic Algorithm

Simulated Annealing

- Occasionally accepts worse solutions.
- Escapes local maxima.
- Probability decreases over time.

Advantages:

- Better exploration.
- Avoids getting stuck.

Genetic Algorithm

Inspired by natural evolution.

Steps:

- Create population.
- Evaluate fitness.
- Select best solutions.
- Crossover.
- Mutation.
- Repeat.

Advantages:

- Explores many solutions simultaneously.
- Handles large search spaces.
- Adapts to changing traffic.

iv. Recommended Approach

Genetic Algorithm

Justification

- Highly scalable.
- Handles dynamic traffic.
- Produces near-optimal solutions.
- Efficient for hundreds of intersections.
- Supports continuous optimization.

3. Autonomous Mars Rover

i. Why Online Search Agent?

Offline search assumes the complete environment is known.

Mars rover:

- Unknown terrain.
- Hazards discovered during movement.
- Limited sensors.

Therefore, the rover must make decisions while exploring.

ii. Challenges

Partial Observability

- Cannot see the complete terrain.
- Hidden obstacles.

Dynamic Environment

- Dust storms.
- Loose rocks.
- New hazards.

Limited Sensors

- Incomplete information.
- Sensor uncertainty.

iii. Internal Model

The rover maintains a map.

Steps:

- Sense surroundings.
- Detect obstacles.
- Update map.
- Mark safe and unsafe regions.
- Plan the next movement.
- Repeat continuously.

iv. Comparison

Feature	Uninformed Search	Informed Search	Online Search
Environment Known	Yes	Yes	No
Uses Heuristic	No	Yes	Optional
Learns During Search	No	No	Yes
Suitable for Mars	No	No	Yes

v. Recommended Approach

Online Search Agent

Reasons:

- Learns while exploring.
- Handles uncertainty.
- Adapts to new obstacles.
- Ensures safer navigation.

4. University Exam Timetable (CSP)

CSP Formulation

Variables

Each subject represents one variable.

Example:

- Math
- Physics
- Chemistry
- English

Domains

Possible exam slots.

Example:

- Monday Morning
- Monday Afternoon
- Tuesday Morning
- Tuesday Afternoon

Constraints

1. No student has overlapping exams.
2. Limited exam halls.
3. Faculty availability.
4. Subject order constraints.
5. Special accommodations.

ii. Backtracking Search

Steps

1. Select a subject.
2. Assign a time slot.
3. Check constraints.
4. If constraints fail, backtrack.
5. Try another slot.
6. Continue until all exams are assigned.

iii. Constraint Propagation (Forward Checking)

Forward checking removes invalid future assignments.

Example:

If Math is assigned Monday Morning,
that slot is removed from subjects sharing students.

Advantages:

- Detects conflicts early.
- Reduces search space.
- Speeds up scheduling.

iv. Real-world Challenges

- Thousands of students.
- Hundreds of courses.
- Faculty availability changes.
- Room shortages.
- Last-minute updates.

Best Strategy

Backtracking + Forward Checking

Reasons:

- Efficient.
- Reduces unnecessary searches.
- Easily scales to large universities.

5. Strategic Game AI (Minimax & Alpha-Beta Pruning)

i. Minimax Algorithm

Minimax assumes:

- AI tries to maximize its score.
- Opponent tries to minimize AI's score.

Steps:

1. Generate game tree.
2. Assign utility values.
3. Alternate MAX and MIN levels.
4. Choose the move with the highest guaranteed score.

ii. Computational Challenges

Problems:

- Huge branching factor.
- Very deep trees.
- High memory usage.
- Slow decision making.

Time Complexity:

$O(b^d)$

where:

- **b** = Branching factor
- **d** = Depth

iii. Alpha-Beta Pruning

Alpha-Beta removes branches that cannot affect the final decision.

Alpha

Best value for MAX.

Beta

Best value for MIN.

When $\alpha \geq \beta$, remaining branches are ignored.

Advantages:

- Evaluates fewer nodes.
- Produces the same optimal result.
- Speeds up search.

Best-case complexity:

$O(b^{(d/2)})$

iv. Evaluation Function

Example:

Evaluation Score =

- Material advantage (+50%)
- Unit health (+20%)
- Resource control (+15%)
- Territory control (+10%)
- Tactical position (+5%)

Higher score indicates a better game state.

Factors Considered

- Number of units.
- Unit health.
- Resources.
- Map control.
- Strategic position.

v. Recommended Strategy

Minimax + Alpha-Beta Pruning + Evaluation Function

Justification

- Produces strong decisions.
- Reduces computation.
- Meets real-time constraints.
- Maintains near-optimal gameplay.
- Widely used in modern game AI.